LAB 3 — Operación segura y observable

Seguridad + observabilidad: detectar, investigar y responder con evidencia

Computación en Nube · EAFIT · Sesión 14 · 13 de octubre de 2026

Duración	Modalidad	Entorno	Carácter
100–110 minutos	Equipos de 4	Cuenta AWS personal · us-east-1	Evaluativo · 10 %

1. Propósito y resultado

Construir una carga sencilla de Digital Café Luna que permita observar un estado normal, provocar un error HTTP 500 controlado, detectar su señal en CloudWatch, investigar la evidencia y comprobar la recuperación. La práctica integra seguridad, logs, métricas, alarma y una vista operacional sin agregar componentes que no aporten al objetivo.

Pregunta rectora: ¿Cómo detectamos, investigamos y respondemos a un problema operacional de Digital Café Luna usando evidencia del sistema?

Usuario / curl	→	EC2 · app HTTP mínima	→	/var/log/dcl/app.log
		IAM Role / Instance Profile		↓
		amazon-cloudwatch-agent	→	CloudWatch Logs
				↓
		CloudWatch Dashboard	←	Metric Filter · ErrorCount
			←	CloudWatch Alarm · OK <-> ALARM

Figura 1. Arquitectura de LAB 3. Una EC2 produce logs de aplicación; el agente unificado los envía a CloudWatch Logs; un Metric Filter genera ErrorCount; la alarma y el dashboard hacen visible el cambio operacional.

OJO TÉCNICO. La práctica demuestra una cadena de observabilidad, no alta disponibilidad. Una sola EC2 es intencional: permite concentrarse en identidad, telemetría, detección, investigación y recuperación. No agregue ALB, ASG, SQS, Lambda, EventBridge ni SSH.

Objetivos de aprendizaje

- Usar IAM Role / Instance Profile y credenciales temporales en lugar de access keys locales.
- Relacionar logs, una métrica derivada y una alarma con un evento real del workload.
- Interpretar la secuencia OK → ALARM → OK mediante evidencia temporal.
- Formular una hipótesis básica a partir de logs y señal operacional.
- Aplicar mínimo privilegio, IMDSv2, retención y cleanup en una práctica observable.

2. Baseline, recursos y convenciones

LAB 3 integra la fase de seguridad trabajada desde S12 con observabilidad y simulación de evento desarrolladas en S13. En S14 se ejecuta la evaluación completa. No depende técnicamente de LAB 2 ni de una persistencia de S09.

Use recursos exclusivos e identificables. Etiquetas sugeridas: dcl:project=digital-cafe-luna, dcl:environment=dev, dcl:owner-team=team-XX y dcl:managed-by=console.

Tipo	Recurso	Nombre / configuración
Reutilizar	VPC / subnet pública	Baseline disponible · us-east-1 · salida a Internet
Crear	Security Group	dcl-dev-sg-lab3 · HTTP/80 desde origen de prueba · sin SSH
Crear	IAM Role / Instance Profile	dcl-dev-role-lab3-ops / dcl-dev-profile-lab3-ops
Crear	EC2	dcl-dev-ops-web · Amazon Linux 2023 · IMDSv2 required
Crear	Aplicación	HTTP mínima · / · /health · /simulate-error
Crear	Log Group	/dcl/dev/ops · retención 7 días

Crear	Metric Filter	patrón ERROR → DigitalCafeLuna/LAB3 · ErrorCount
Crear	Alarm	dcl-dev-ops-alarm · Sum · 60 s · >=1 · 1/1 · missing=notBreaching
Crear	Dashboard	dcl-dev-ops-dashboard · ErrorCount + estado de alarma

OJO TÉCNICO. defaultValue=0 no significa “cero continuo”. El preflight observó ErrorCount=0 cuando hubo logs normales ingeridos sin coincidencia ERROR. Si un período no contiene datapoints, la alarma resuelve esa ausencia mediante TreatMissingData=notBreaching.

3. Implementación paso a paso

Fase 0 — Verificar región, red y alcance

Qué hacemos: Confirmar el entorno real antes de crear recursos.

Por qué: El laboratorio debe crear únicamente lo necesario y evitar depender de infraestructura inexistente.

1. Confirme la Región us-east-1.

2. Identifique una VPC y una subnet pública con ruta 0.0.0.0/0 hacia un Internet Gateway.

3. Verifique que la EC2 pueda recibir IPv4 pública para la prueba HTTP.

4. No reconstruya recursos de sesiones anteriores que no sean necesarios.

Validación: Existe una ubicación válida para una EC2 con HTTP público controlado y salida hacia los endpoints de AWS.

Fase 1 — Crear Log Group y retención

Qué hacemos: Preparar el destino explícito de los logs antes de lanzar el workload.

Por qué: Crear el Log Group desde la identidad del estudiante permite reducir permisos del Instance Profile.

1. Abra CloudWatch > Logs > Log groups > Create log group.

2. Name: /dcl/dev/ops.

3. Configure Retention = 7 days.

4. Registre la Región us-east-1 y confirme que el grupo está vacío inicialmente.

Validación: /dcl/dev/ops existe con retención de 7 días.

Fase 2 — Crear IAM Role / Instance Profile de mínimo privilegio

Qué hacemos: Dar al agente una identidad temporal para escribir exclusivamente la telemetría requerida.

Por qué: El workload no necesita access keys locales ni permisos administrativos sobre CloudWatch.

1. Cree un IAM Role para el servicio EC2: dcl-dev-role-lab3-ops. Cree explícitamente el Instance Profile dcl-dev-profile-lab3-ops y agregue a este el role dcl-dev-role-lab3-ops.

2. Cree una policy inline o administrada por el equipo, sustituyendo ACCOUNT_ID por el ID real de la cuenta:

```
{
 "Version": "2012-10-17",
 "Statement": [
  {
   "Sid": "WriteLabLogs",
   "Effect": "Allow",
   "Action": [
    "logs:CreateLogStream",
    "logs:DescribeLogStreams",
    "logs:PutLogEvents"
   ],
```

```
    "Resource": "arn:aws:logs:us-east-1:ACCOUNT_ID:log-group:/dcl/dev/ops:*"
  }
 ]
}
```

3. Asocie la policy al role y confirme que la trust policy permite sts:AssumeRole al principal de servicio ec2.amazonaws.com.

4. Al lanzar la instancia, seleccione el Instance Profile dcl-dev-profile-lab3-ops.

OJO TÉCNICO. No use CloudWatchFullAccess. El role del workload solo necesita escribir sus logs. Crear el Log Group, Metric Filter, alarma y dashboard corresponde a la identidad con la que el estudiante administra el laboratorio, no al Instance Profile.

Validación: El role no contiene permisos SQS, S3, Lambda ni privilegios administrativos ajenos a LAB 3.

Fase 3 — Crear Security Group y lanzar EC2

Qué hacemos: Desplegar un único workload HTTP con superficie de red mínima.

Por qué: Una sola EC2 basta para producir la señal que estudiaremos.

1. Cree dcl-dev-sg-lab3 en la VPC seleccionada.

2. Inbound: HTTP TCP/80 únicamente desde el origen autorizado para la prueba. No agregue SSH/22.

3. Lance dcl-dev-ops-web con Amazon Linux 2023 y un tipo pequeño apropiado.

4. Seleccione subnet pública, IPv4 pública y dcl-dev-sg-lab3.

5. En Advanced details, asocie el Instance Profile dcl-dev-profile-lab3-ops y establezca Metadata version = V2 only / IMDSv2 required.

6. No configure access keys en user data, archivos o variables de entorno.

Validación: La instancia está Running, tiene el role correcto, IMDSv2 requerido y solo HTTP/80 expuesto para la prueba.

Fase 4 — Instalar la aplicación y el agente unificado

Qué hacemos: Preparar una aplicación mínima que registre INFO para tráfico normal y ERROR para el evento controlado, y enviar ese archivo a CloudWatch Logs.

Por qué: La misma evidencia permitirá detectar e investigar el problema.

Use user data al lanzar la instancia para evitar acceso administrativo interactivo. El siguiente bloque instala amazon-cloudwatch-agent, crea la aplicación y la registra como servicio:

```
#!/bin/bash
set -euo pipefail
dnf install -y amazon-cloudwatch-agent
mkdir -p /opt/dcl /var/log/dcl
touch /var/log/dcl/app.log
cat > /opt/dcl/app.py <<'PYAPP'
from http.server import BaseHTTPRequestHandler, HTTPServer
from datetime import datetime, timezone
LOG = "/var/log/dcl/app.log"
def write(level, path, status, message):
    ts = datetime.now(timezone.utc).isoformat()
    with open(LOG, "a", encoding="utf-8") as f:
        f.write(f"{ts} {level} path={path} status={status} message={message}\n")
class Handler(BaseHTTPRequestHandler):
    def do_GET(self):
        if self.path == "/simulate-error":
            status, level, msg = 500, "ERROR", "controlled_failure"
        elif self.path in ("/", "/health"):
            status, level, msg = 200, "INFO", "request_ok"
        else:
            status, level, msg = 404, "INFO", "not_found"
```

```
        write(level, self.path, status, msg)
        body = f"Digital Cafe Luna | {status} | {msg}\n".encode()
        self.send_response(status); self.send_header("Content-Type","text/plain")
        self.send_header("Content-Length", str(len(body))); self.end_headers()
        self.wfile.write(body)
    def log_message(self, format, *args):
        return
HTTPServer(("0.0.0.0", 80), Handler).serve_forever()
PYAPP
cat > /etc/systemd/system/dcl-app.service <<'UNIT'
[Unit]
Description=Digital Cafe Luna LAB3 app
After=network-online.target
[Service]
Type=simple
ExecStart=/usr/bin/python3 /opt/dcl/app.py
Restart=on-failure
[Install]
WantedBy=multi-user.target
UNIT
cat > /opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.json <<'CW'
{
  "agent": {"run_as_user": "root"},
  "logs": {"logs_collected": {"files": {"collect_list": [{
    "file_path": "/var/log/dcl/app.log",
    "log_group_name": "/dcl/dev/ops",
    "log_stream_name": "{instance_id}"
  }]}}}
}
CW
systemctl daemon-reload
systemctl enable --now dcl-app
/opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-ctl \
  -a fetch-config -m ec2 \
  -c file:/opt/aws/amazon-cloudwatch-agent/etc/amazon-cloudwatch-agent.json -s
```

Validación: http://PUBLIC_IP/ y http://PUBLIC_IP/health responden HTTP 200. En CloudWatch Logs aparece un stream bajo /dcl/dev/ops con eventos INFO.

Fase 5 — Verificar identidad y telemetría normal

Qué hacemos: Capturar el estado normal antes de provocar el evento.

Por qué: Sin baseline no podemos demostrar cambio ni recuperación.

1. En EC2 > Instances > dcl-dev-ops-web, capture evidencia del IAM Role asociado y Metadata version = V2 only.

2. Ejecute varias solicitudes a /health desde su equipo o CloudShell:

```
curl -i http://PUBLIC_IP/health
curl -i http://PUBLIC_IP/
```

3. Abra CloudWatch Logs > /dcl/dev/ops y confirme eventos INFO con status=200.

Validación: workload normal + identidad por role + logs INFO reales. No existen access keys locales.

Fase 6 — Crear Metric Filter ErrorCount

Qué hacemos: Convertir cada evento de log que contenga ERROR en una unidad de la métrica ErrorCount.

Por qué: Una señal cuantitativa permite establecer una condición de alarma sobre el comportamiento real de la aplicación.

1. Abra /dcl/dev/ops > Metric filters > Create metric filter.
2. Filter pattern: ERROR.
3. Filter name: dcl-dev-error-filter.
4. Metric namespace: DigitalCafeLuna/LAB3.
5. Metric name: ErrorCount.
6. Metric value: 1.
7. Default value: 0. No configure dimensiones.
8. Cree el filtro.

OJO TÉCNICO. El cero solo se publica cuando durante el período se ingieren logs y ninguno coincide con ERROR. Si no hay ingesta/datapoints, defaultValue no crea por sí solo un cero. Esta diferencia importa al interpretar la alarma.

9. Genere tráfico normal con /health y espere la aparición de ErrorCount. En Metrics > All metrics > DigitalCafeLuna/LAB3 seleccione Statistic = Sum.

Validación: actividad normal sin ERROR produce ErrorCount Sum=0, tal como se verificó en preflight.

Fase 7 — Crear la alarma

Qué hacemos: Crear una condición capaz de cambiar realmente entre OK y ALARM.

Por qué: Una alarma útil debe responder a una señal generada por el sistema, no existir solo como configuración decorativa.

1. Desde la métrica ErrorCount, seleccione Create alarm.
2. Statistic = Sum; Period = 1 minute (60 seconds).
3. Threshold type = Static.
4. Whenever ErrorCount is Greater/Equal (>=) 1.
5. Evaluation periods = 1 y Datapoints to alarm = 1.
6. Missing data treatment = Treat missing data as good (not breaching).
7. Alarm name = dcl-dev-ops-alarm.
8. No agregue SNS: la notificación externa no es requisito de LAB 3. Cree la alarma y espere estado OK.

Validación: dcl-dev-ops-alarm alcanza OK antes de ejecutar el evento.

Fase 8 — Crear dashboard mínimo

Qué hacemos: Reunir la señal y el estado de alarma en una vista operacional pequeña.

Por qué: Un dashboard útil prioriza información accionable, no cantidad de widgets.

1. Cree CloudWatch Dashboard dcl-dev-ops-dashboard.
2. Agregue un widget Line / time series para DigitalCafeLuna/LAB3 > ErrorCount, Statistic = Sum, Period = 1 minute.
3. Agregue un widget Alarm status para dcl-dev-ops-alarm.
4. Guarde el dashboard.

Validación: La vista muestra ErrorCount y el estado actual de la alarma.

4. Validación funcional: ciclo operacional

Prueba A — Estado normal

1. Genere varias solicitudes normales:

```
curl -s -o /dev/null -w "%{http_code}\n" http://PUBLIC_IP/health
curl -s -o /dev/null -w "%{http_code}\n" http://PUBLIC_IP/
```

2. Confirme HTTP 200, logs INFO, ErrorCount Sum=0 cuando hubo actividad normal sin coincidencia ERROR y alarma OK.

Checkpoint: Capture dashboard y logs normales antes del evento.

Prueba B — Evento HTTP 500 controlado

Qué hacemos: Provocar un error seguro y reversible sin detener ni modificar infraestructura.

1. Registre T0 y ejecute una sola solicitud de error controlado:

```
date -u +"%Y-%m-%dT%H:%M:%SZ"
curl -i http://PUBLIC_IP/simulate-error
```

2. Confirme HTTP 500. No repita el evento hasta observar la señal del primer error.
3. En /dcl/dev/ops localice el evento ERROR con path=/simulate-error y status=500. Registre T1 cuando sea visible.
4. En Metrics confirme ErrorCount Sum=1. Registre T2 cuando el datapoint sea visible.
5. Observe dcl-dev-ops-alarm hasta ALARM y registre T3.
6. Capture el dashboard mostrando ErrorCount y ALARM.

OJO TÉCNICO. CloudWatch no es instantáneo. No califique como fallo que T1, T2 o T3 aparezcan con retraso. La evidencia debe conservar timestamps y la secuencia causal, no asumir latencias exactas.

Validación: un único error controlado produce log ERROR, ErrorCount Sum=1 y transición OK → ALARM.

Prueba C — Investigación básica

Qué hacemos: Usar la señal para formular una hipótesis basada en evidencia.

1. Parta del estado ALARM y ubique el intervalo temporal del datapoint ErrorCount.
2. Revise /dcl/dev/ops en el mismo intervalo.
3. Identifique path=/simulate-error, status=500 y message=controlled_failure.
4. Formule una hipótesis breve que relacione alarma, métrica y evento observado.

Resultado esperado: La alarma coincide temporalmente con un error HTTP 500 controlado registrado por el workload.

Prueba D — Recuperación y retorno a OK

Qué hacemos: Cesar el evento y demostrar que el sistema vuelve a su condición normal.

1. No invoque nuevamente /simulate-error.
2. Genere tráfico normal con /health durante el siguiente período de evaluación.
3. Confirme nuevos eventos INFO y HTTP 200.
4. Observe ErrorCount Sum=0 cuando existe ingesta normal sin coincidencia ERROR.
5. Espere la transición ALARM → OK y registre la hora.

Validación: workload recuperado, evidencia normal y alarma nuevamente OK.

Prueba E — Reproducibilidad

Repita una segunda vez la secuencia completa: OK → /simulate-error → ERROR → ErrorCount=1 → ALARM → investigación → /health → OK. El segundo ciclo confirma que el comportamiento no dependió de una única ejecución accidental.

Validación: Dos ciclos completos y coherentes quedan documentados.

5. Evidencias y criterios de aceptación

El entregable debe mostrar una secuencia causal. Una colección de capturas de recursos creados, sin relación temporal entre ellas, no demuestra observabilidad.

Comprobación	Resultado esperado
Arquitectura	Una EC2 + IAM Role/Profile + app + agente unificado + Logs + Metric Filter + Alarm + Dashboard.

Seguridad	IMDSv2 required; sin access keys locales; SG solo HTTP/80 para la prueba; sin SSH.
Normal	/ y /health → 200; INFO visible; ErrorCount=0 con actividad normal; Alarm=OK.
Evento	/simulate-error → 500 controlado y evento ERROR visible.
Señal	Metric Filter ERROR produce ErrorCount Sum=1 para el error observado.
Alarma	60 s · >=1 · 1/1 · missing=notBreaching; transición OK → ALARM.
Diagnóstico	El equipo correlaciona alarma, métrica y log para formular una hipótesis.
Recuperación	Tráfico normal; retorno ALARM → OK.
Reproducibilidad	Dos ciclos completos documentados.
Cleanup	No quedan recursos activos exclusivos de LAB 3.

Registro temporal mínimo

Ciclo	T0 evento	T1 log visible	T2 métrica visible	T3 ALARM	Retorno a OK
1					
2					

No existe un tiempo “correcto” prefijado para T1–T3. Registre lo observado y explique cualquier retraso o diferencia entre ciclos.

6. Entregable y rúbrica

Entregue un documento breve y verificable con los diez componentes comunes del curso:

1. Objetivo y arquitectura.
2. Recursos implementados.
3. Procedimiento resumido.
4. Validaciones.
5. Evidencias.
6. Problemas y soluciones.
7. Seguridad y costos.
8. Cleanup.
9. Contribuciones.
10. Retrospectiva.

Criterio	Peso	Evidencia principal en LAB 3
Implementación técnica	30 %	Workload, IAM, agente, Logs, Metric Filter, Alarm y Dashboard funcionales.
Validación funcional	20 %	Dos ciclos OK → ALARM → OK con señal y recuperación observables.
Arquitectura y buenas prácticas	15 %	Arquitectura mínima, señal coherente y decisiones justificadas.
Seguridad, costos y cleanup	10 %	Role/Profile, IMDSv2, sin claves, SG mínimo, retención y cleanup completo.
Evidencias y documentación	15 %	T0–T3, logs, métrica, alarma, dashboard e hipótesis correlacionados.
Colaboración y defensa	10 %	Contribuciones identificadas y explicación técnica por cualquier integrante.

7. Costos y cleanup obligatorio

LAB 3 utiliza recursos con costos potenciales en una cuenta AWS personal. El costo depende de región, tipo de instancia, volumen, IPv4 pública, ingestión/retención de logs, métricas, alarmas y dashboards. Consulte precios vigentes antes de ejecutar y elimine los recursos exclusivos al finalizar.

Capture primero las evidencias. Después realice cleanup en este orden práctico:

1. Termine dcl-dev-ops-web y espere estado Terminated.

2. Elimine dcl-dev-sg-lab3 cuando no tenga interfaces asociadas.

3. Elimine dcl-dev-ops-alarm.

4. Elimine dcl-dev-ops-dashboard.

5. Elimine el Metric Filter dcl-dev-error-filter.

6. Elimine el Log Group /dcl/dev/ops. Esto elimina también sus streams y datos retenidos.

7. Desasocie y elimine dcl-dev-profile-lab3-ops; luego elimine dcl-dev-role-lab3-ops junto con su policy exclusiva.

8. Verifique EC2, CloudWatch Logs, Alarms, Dashboards e IAM para confirmar que no queden recursos exclusivos activos.

OJO TÉCNICO. Las métricas personalizadas pueden permanecer visibles en CloudWatch durante un tiempo aunque ya no reciban datapoints; no existe una operación para borrar manualmente una métrica. El cleanup se considera completo cuando se eliminaron los recursos configurables que la producen o consumen y no quedan recursos activos exclusivos.

8. Cierre técnico

LAB 3 completa una cadena operacional sencilla pero completa: identidad segura → telemetría → señal → alarma → investigación → recuperación. El objetivo no es memorizar pantallas de CloudWatch, sino demostrar que una hipótesis operacional puede sostenerse con evidencia del sistema.

Cadena final: estado normal → evento HTTP 500 controlado → log ERROR → ErrorCount → ALARM → investigación → tráfico normal → OK.

Frontera: no se requieren SIEM, GuardDuty como eje, AIOps, tracing distribuido avanzado, autoscaling avanzado, performance tuning, FinOps profundo, Terraform, Lambda/EventBridge ni arquitecturas distribuidas adicionales.

Fuentes oficiales

- Amazon CloudWatch Agent — Collect metrics, logs, and traces with the CloudWatch agent.
- Amazon CloudWatch Logs — Create metrics from log events using filters.
- Amazon CloudWatch — Using Amazon CloudWatch alarms.
- Amazon CloudWatch — Treating missing data in a CloudWatch alarm.
- Amazon EC2 — Configure the instance metadata options / IMDSv2.
- AWS IAM — IAM roles for Amazon EC2.
- Amazon CloudWatch pricing — consultar valores vigentes antes de la ejecución.